

Grafos

1. Definição de um Grafo

Um grafo é um conjunto de objetos chamados **vértices**, que podem estar interligados por **arestas**. Exemplos:

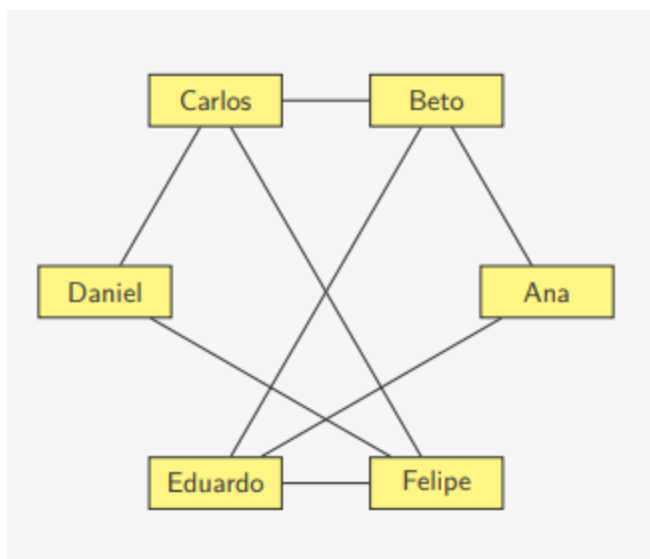


Figura 1: Rede social representada como um grafo.

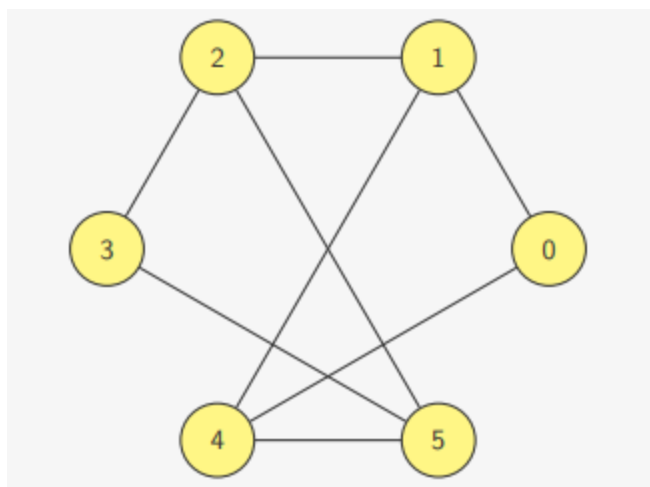


Figura 2: Grafo não direcionado.

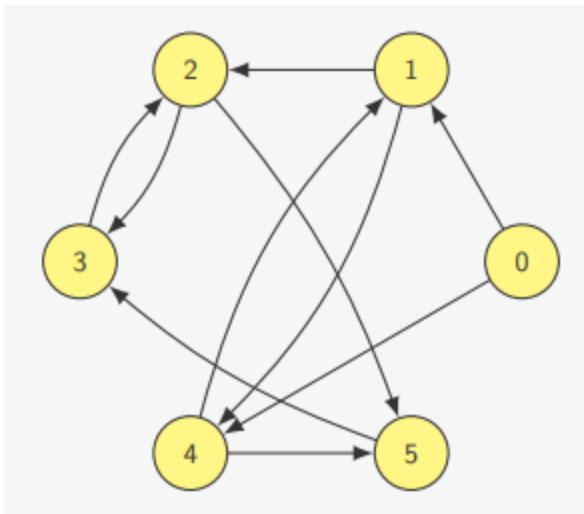


Figura 3: Grafo direcionado.

1.1. Representação Matemática

Matematicamente, um grafo é um par ordenado $(G = (V, A))$, onde:

- (V) é o conjunto de vértices. Exemplo:

$$V = \{0, 1, 2, 3, 4, 5\}$$

- (A) é o conjunto de arestas. Para um grafo não direcionado:

$$A = \{\{0, 1\}, \{0, 4\}, \{1, 4\}, \{1, 2\}, \{2, 5\}, \{2, 3\}, \{3, 5\}, \{4, 5\}\}$$

Para um grafo direcionado, as arestas são pares ordenados:

$$A = \{(0, 1), (1, 4), (1, 2), (2, 5), (2, 3), (3, 2), (4, 1), (4, 5), (5, 3)\}$$

1.2. Adjacência

No grafo da **Figura 2**, o vértice 0 é vizinho do vértice 4. Assim:

- Dizemos que 0 e 4 são **adjacentes**.
- A **vizinhança** do vértice 4 é composta pelos vértices 0, 1 e 5.

1.3. Grau do Vértice

O **grau** de um vértice (v) é a quantidade de arestas incidentes a ele, ou seja, o número de vértices em sua vizinhança.

2. Formas de Representação

2.1. Matriz de Adjacências

Uma forma de representar grafos é por meio de uma **matriz de adjacências**, seguindo estas regras:

- Se o grafo tem (n) vértices, a matriz terá dimensão ($n \times n$).
- Cada vértice é numerado de 0 a $n-1$.
- Se dois vértices são vizinhos, a matriz recebe 1, caso contrário, recebe 0.
 - (adjacencia[u][v] = 1) → u e v são vizinhos.
 - (adjacencia[u][v] = 0) → u e v não são vizinhos.

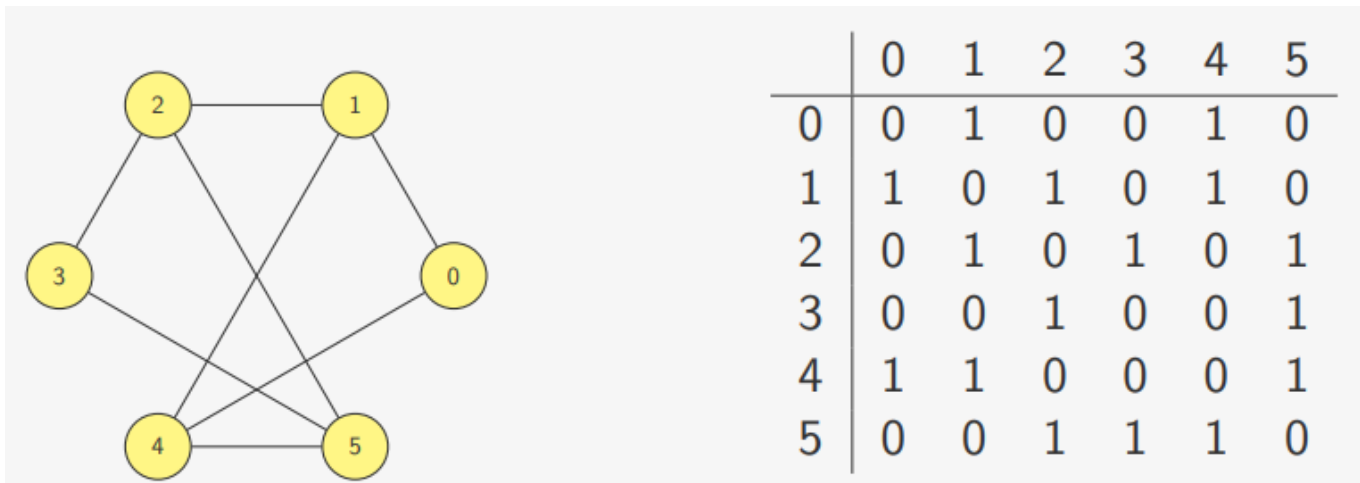


Figura 4: Representação de um grafo não direcionado em uma matriz de adjacências.



Figura 5: Representação de grafos direcionado e não direcionado.

2.1.1. Implementação em C

2.1.1.1. Estrutura Base

```
typedef struct {
    int **adj;
    int n; // Número de vértices
} grafo;

grafo *criaGrafo(int n) {
    grafo *g = malloc(sizeof(grafo));
    g->adj = malloc(n * sizeof(int*));

    for (int i = 0; i < n; i++) {
        g->adj[i] = calloc(n, sizeof(int));
    }

    g->n = n;
    return g;
}
```

2.1.1.2. Insere

```
void insereAresta(int i, int j, grafo *g){

    g->adj[i][j] = 1;
    g->adj[j][i] = 1; // Se não for direcionado
}
```

2.1.1.3. Remove

```
void removeAresta(int i, int j, grafo *g){

    g->adj[i][j] = 0;
    g->adj[j][i] = 0; // Se não for direcionado
}
```

2.1.1.4. Cálculo do Grau de um Vértice

```
int grau(grafo *g, int v){
    int gr = 0;

    for(int i=0; i < g->n; i++){
        gr+=g->adj[v][i];
    }
}
```

```

    return gr;

}

```

2.1.1.4. Recomendações

```

void recomendacoes(grafo *g, int v){
    int u, w;
    for (u = 0; u < g->n; u++){
        if(g->adj[v][u]){
            for(w=0;w<g->n;w++){
                if(g->adj[u][w] && w != v && !g->adj[v][w]){
                    printf("%d\n", w);
                }
            }
        }
    }
}

```

2.1.2. Vantagens

- A verificação da existência de uma aresta entre dois vértices ocorre em tempo constante $O(1)$, pois basta acessar a posição correspondente na matriz.
- Implementação mais simples, especialmente para algoritmos que envolvem operações matriciais.
- Útil para grafos densos, onde o número de arestas é próximo ao número máximo possível $n(n - 1)/2$ para grafos não direcionados.

No entanto, a matriz de adjacências pode consumir muita memória, especialmente para grafos esparsos, devido à necessidade de armazenar valores nulos para arestas inexistentes.

2.2. Lista de adjacências

Outra forma eficiente de representar grafos é por meio de uma **lista de adjacências**, seguindo estas regras:

- Se o grafo tem n vértices, a lista contém n elementos.
- Cada elemento da lista corresponde a um vértice e armazena uma lista de seus vizinhos.
- Se dois vértices são vizinhos, a lista do primeiro contém o segundo e vice-versa (caso o grafo não seja direcionado).
- $v \in lista[u] \rightarrow u$ e v são vizinhos.

- $v \notin lista[u] \rightarrow u$ e v não são vizinhos.

Essa abordagem é geralmente mais eficiente em termos de memória, especialmente para grafos esparsos, pois evita armazenar grande quantidade de zeros, como acontece na matriz de adjacências.

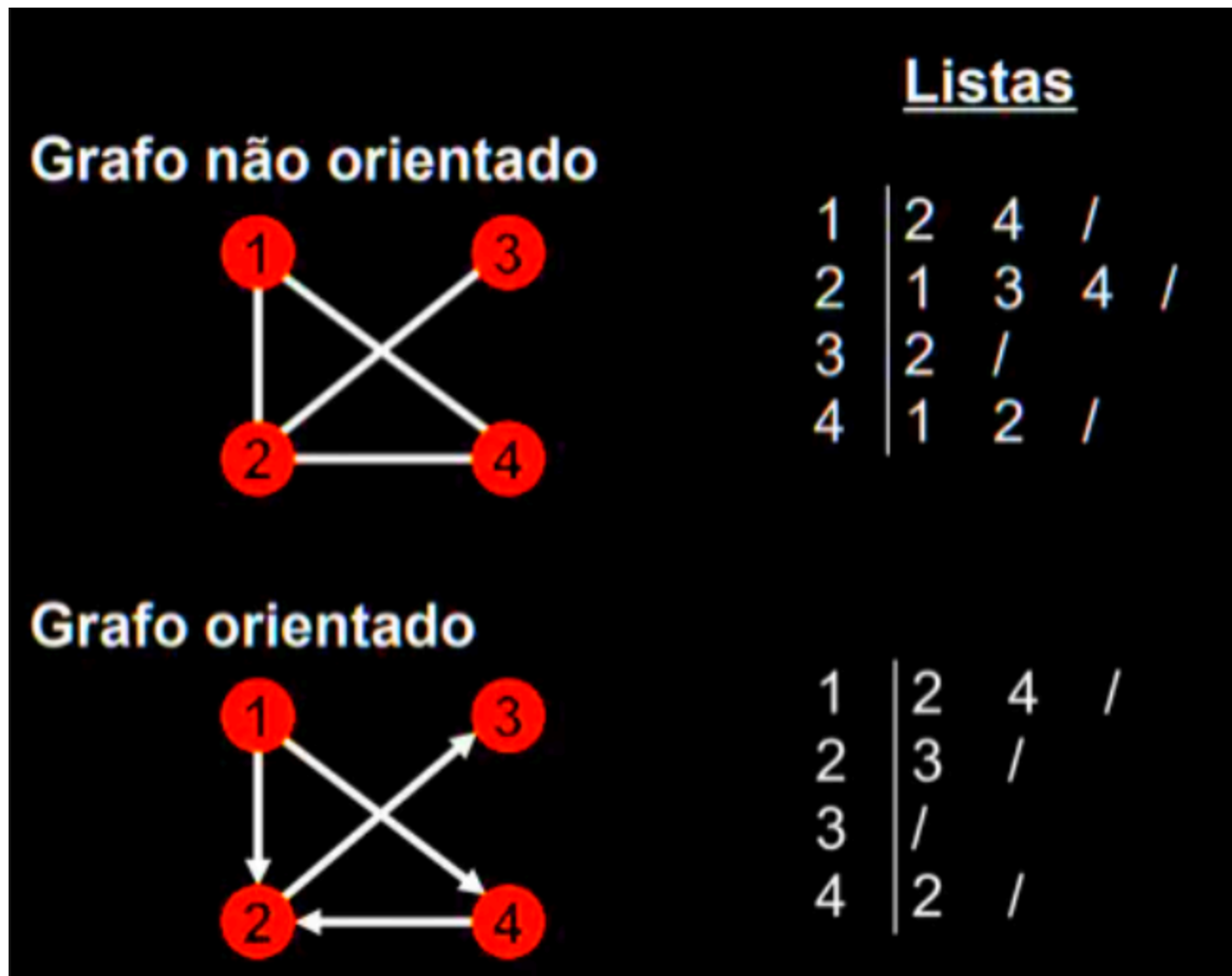


Figura 6: Demonstrações dos grafos dirigido e não dirigido aplicado em uma lista de adjacência.

2.2.1. Implementação em C

2.2.1.1. Estrutura Base

```

#include <stdio.h>
#include <stdlib.h>

typedef struct No {
    int vertice;
    struct No *prox;
} No;

```

```

typedef struct {
    No **lista;
    int n;
} Grafo;

Grafo *criaGrafo(int n) {
    Grafo *g = malloc(sizeof(Grafo));
    g->lista = malloc(n * sizeof(No*));

    for (int i = 0; i < n; i++) {
        g->lista[i] = NULL;
    }

    g->n = n;
    return g;
}

```

2.2.1.2. Insere

```

void insereAresta(Grafo *g, int i, int j) {
    No *novo = malloc(sizeof(No));
    novo->vertice = j;
    novo->prox = g->lista[i];
    g->lista[i] = novo;

    // Se não for direcionado, adiciona a aresta inversa
    novo = malloc(sizeof(No));
    novo->vertice = i;
    novo->prox = g->lista[j];
    g->lista[j] = novo;
}

```

2.2.1.3. Remove

```

void removeAresta(Grafo *g, int i, int j) {
    No *atual = g->lista[i], *anterior = NULL;
    while (atual && atual->vertice != j) {
        anterior = atual;
        atual = atual->prox;
    }
    if (atual) {
        if (anterior)
            anterior->prox = atual->prox;
    }
}

```

```

        else
            g->lista[i] = atual->prox;
        free(atual);
    }

    // Se não for direcionado, remove a aresta inversa
    atual = g->lista[j], anterior = NULL;
    while (atual && atual->vertice != i) {
        anterior = atual;
        atual = atual->prox;
    }
    if (atual) {
        if (anterior)
            anterior->prox = atual->prox;
        else
            g->lista[j] = atual->prox;
        free(atual);
    }
}

```

2.2.1.4. Cálculo do Grau de um Vértice

```

int grau(Grafo *g, int v) {
    int gr = 0;
    No *atual = g->lista[v];
    while (atual) {
        gr++;
        atual = atual->prox;
    }
    return gr;
}

```

2.2.1.5. Recomendações

```

void recomendacoes(Grafo *g, int v) {
    No *u, *w;
    for (u = g->lista[v]; u != NULL; u = u->prox) {
        for (w = g->lista[u->vertice]; w != NULL; w = w->prox) {
            if (w->vertice != v && !pertence(g->lista[v], w->vertice)) {
                printf("%d\n", w->vertice);
            }
        }
    }
}

```



```

int pertence(No *lista, int v) {
    while (lista) {
        if (lista->vertice == v) return 1;
        lista = lista->prox;
    }
    return 0;
}

```

2.2.2. Vantagens

A lista de adjacências é mais vantajosa em grafos esparsos, pois economiza memória e permite percorrer vizinhos de um vértice de maneira eficiente. No entanto, pode ser menos eficiente para verificar diretamente a existência de uma aresta entre dois vértices, pois pode exigir uma busca na lista de adjacências.

3. Desempenho de cada implementação

Desempenho	Vetor de Arcos	Matriz de Adjacências	Lista de Adjacências
Espaço	$O(E)$	$O(V^2)$	$O(V + E)$
Inicialização	$O(1)$	$O(V^2)$	$O(V)$
Cópia	$O(E)$	$O(V^2)$	$O(E)$
Destruição	$O(1)$	$O(V)$	$O(E)$
Inserir Arco	$O(1)$	$O(1)$	$O(V)$
Encontrar Arco	$O(E)$	$O(1)$	$O(V)$
Remover Arco	$O(E)$	$O(1)$	$O(V)$